

Python for Robotics

DR. TNOURJI, ABDELLAH

Research Scientist

The UM6P Vanguard Center

Autonomous Drone Engineering Internship

Mohammed VI Polytechnic University, UM6P , September 15, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

1 Python Refresher for Robotics

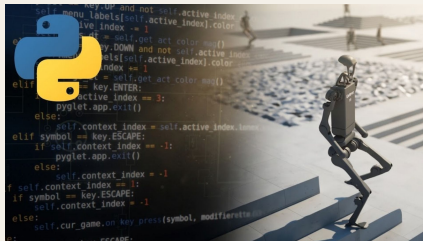
1 Python Refresher for Robotics

- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

Introduction

A robotics program transforms inputs into useful outputs

Sensors and configuration → Python logic → telemetry, commands, logs, and detections



1 Python Refresher for Robotics

■ Core Language Foundations

- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

Variables and Data Types

Names that refer to values used by the robot

Common Types

- ▶ `int`: number of detections
- ▶ `float`: altitude in metres
- ▶ `bool`: mission enabled or disabled
- ▶ `str`: frame or object name
- ▶ `None`: value not available yet

```
altitude_m = 4.0
detection_count = 3
mission_active = True
target_label = "inspection_car"
latest_position = None
```

Robotics Habit

Include units in names such as `altitude_m`, `time_s`, and `rate_hz`. Python is dynamically typed, but engineering meaning must remain explicit.

Lists, Tuples, and Dictionaries

Choose a container according to the meaning of the data

Intuitive Roles

- ▶ **List:** ordered collection that may change
- ▶ **Tuple:** fixed ordered record
- ▶ **Dictionary:** named key-value lookup

```
ranges_m = [2.4, 2.1, 1.9]
position_ned = (3.0, -1.0, -4.0)
state = {
    "mode": "ORBIT",
    "battery_pct": 78.0
}
```

Selection Rule

Use a tuple for one position, a list for a history of measurements, and a dictionary when names such as `mode` or `battery_pct` make the record clearer.

Indexing, Slicing, and Unpacking

Access exactly the data needed by the algorithm

```
position_ned = (3.0, -1.0, -4.0)  
north, east, down = position_ned
```

```
recent_ranges = [2.8, 2.5, 2.2, 2.0]  
latest = recent_ranges[-1]  
last_three = recent_ranges[-3:]
```

Index

Select one element. Python starts counting at zero; index `-1` means the last element.

Unpacking

Give meaningful names to the components of a structured value.

Mutability and Copies

Two names may refer to the same changing object

```
measurements = [1.0, 2.0]
alias = measurements
alias.append(3.0)
# measurements is now [1.0, 2.0, 3.0]

independent_copy = measurements.copy()
```

Intuition

Unexpected shared changes can corrupt a measurement history, configuration, or message buffer. Use `copy()` when an independent list is required.

Conditions

Choose behavior from validated information

```
if latest_position is None:  
    status = "waiting_for_odometry"  
elif altitude_m > max_altitude_m:  
    status = "limit_exceeded"  
else:  
    status = "ready"
```

Robotics Interpretation

Conditions implement decisions such as waiting for sensor data, rejecting invalid parameters, changing mission phase, or suppressing an unsafe output.

Boundary

An if statement is program logic, not proof of flight safety. Safety requires validated requirements, system-level safeguards, testing, and operational procedures.

For Loops

Apply the same operation to a known collection

```
detections = ["car", "person", "car"]  
car_count = 0  
  
for label in detections:  
    if label == "car":  
        car_count += 1
```

Robotics Uses

Process every detection, inspect each sensor sample, publish a sequence of waypoints, or declare a set of ROS 2 parameters.

Readable Iteration

Iterate directly over values. Use `enumerate(...)` only when an index is genuinely needed.

While Loops and Repeated Callbacks

Repeat until a condition changes, but always define an exit

```
attempt = 0
while attempt < 3:
    attempt += 1
    print(f"Connection attempt {attempt}")
```

Use while When

The number of repetitions depends on a changing condition.

In ROS 2

Timers and callbacks usually provide repetition. Avoid blocking loops inside callbacks.

Failure Mode

An unlimited loop can freeze a node or block message processing. Define a stop condition, timeout, or cancellation path.

Functions

Give one reusable operation a clear name

```
import math

def planar_distance_m(x1, y1, x2, y2):
    """Return 2D Euclidean distance in metres."""
    dx = x2 - x1
    dy = y2 - y1
    return math.hypot(dx, dy)

distance_m = planar_distance_m(0.0, 0.0, 3.0, 4.0)
```

Why Functions Matter

A function isolates one responsibility: calculate distance, validate parameters, convert coordinates, filter detections, or build a message.

Make the function contract visible

Function Contract

Project Connection

U M 6 P

Modules and Imports

Separate the project into focused, reusable files

```
# trajectory.py
def tracking_error(desired, measured):
    ...
```

```
# orbit_controller.py
from .trajectory import tracking_error
```

Module

A Python file containing related functions, classes, and constants.

Package

A directory that organizes related modules under one importable namespace.

Design Rule

Keep trajectory mathematics, ROS 2 node logic, configuration handling, and tests in

Why Object-Oriented Programming?

Keep related state and behavior together

Procedural view: data + separate functions

Object-oriented view: state + behavior inside one object

Class, Object, Attributes, and Methods

The blueprint and the individual instance

```
class DroneState:
    def __init__(self, north_m, east_m, down_m):
        self.north_m = north_m
        self.east_m = east_m
        self.down_m = down_m

    def altitude_m(self):
        return -self.down_m

state = DroneState(2.0, -1.0, -4.0)
print(state.altitude_m())
```

Vocabulary

Class: blueprint. **Object:** one instance. **Attributes:** stored state. **Methods:** operations performed by the object.

Constructor and self

Initialize every object in a valid and understandable state

```
class Detection:
    def __init__(self, label, confidence, center_px):
        self.label = label
        self.confidence = confidence
        self.center_px = center_px
```

`__init__`

Runs during object construction and stores the initial state.

`self`

Refers to the specific object whose method is currently running.

Engineering Habit

Validate constructor inputs when an invalid object would make later behavior ambiguous or unsafe.

Encapsulation and Clear Responsibilities

Expose useful behavior and keep implementation details local

```
class AltitudeMonitor:
    def __init__(self, limit_m):
        self._limit_m = float(limit_m)

    def within_limit(self, altitude_m):
        return 0.0 <= altitude_m <= self._limit_m
```

Python Convention

A leading underscore such as `_limit_m` means “internal implementation detail.” It is a convention, not a security boundary.

Project Connection

A class should have one coherent responsibility: mission state, telemetry filtering, detection processing, or ROS 2 communication.

Inheritance in ROS 2 Nodes

Extend an existing class instead of rebuilding its capabilities

```
from rclpy.node import Node

class OrbitController(Node):
    def __init__(self):
        super().__init__("orbit_controller")
        self.timer = self.create_timer(0.05, self.tick)

    def tick(self):
        self.get_logger().info("Controller active")
```

Meaning

`OrbitController` is a specialized `Node`. Through inheritance it receives ROS 2 node capabilities, while adding project-specific behavior.

Design Restraint

Prefer composition for unrelated capabilities. Use inheritance when the “is a” relationship is clear and supported by the framework.

1 Python Refresher for Robotics

- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

Syntax Errors and Exceptions

Different failures occur at different moments

Syntax Error

Python cannot parse the program.
Typical causes include missing colons, parentheses, or incorrect indentation.

Exception

The program starts, but a runtime operation fails, such as opening a missing file or converting invalid text to a number.

Debugging Order

Read the final exception type and message, then inspect the traceback from the point where your code first appears.

Handling Expected Exceptions

Recover only when the program has a meaningful response

```
try:
    altitude_m = float(raw_altitude)
except ValueError as exc:
    logger.error("Invalid altitude: %s", exc)
    altitude_m = None
```

Good Practice

Catch specific exceptions, preserve useful evidence, and choose a clear fallback such as rejecting configuration, skipping one record, or entering a passive state.

Avoid

Do not use a silent broad `except: .` It can hide programming defects and leave a robotics process running with an unknown state.

Raising Exceptions During Validation

Reject invalid assumptions close to their source

```
def validate_altitude(altitude_m, max_altitude_m):  
    if not 0.5 <= altitude_m <= max_altitude_m:  
        raise ValueError("altitude_m outside limits")
```

Project Connection

Mission parameters should be checked before commands are published. A clear `ValueError` is easier to diagnose than incorrect motion caused by an invalid value.

Principle

Exceptions report exceptional conditions. Normal mission choices should remain explicit conditions and state transitions.

Reading Configuration Files

Separate adjustable settings from program logic

```
import json
from pathlib import Path

config_path = Path("config/mission.json")
with config_path.open("r", encoding="utf-8") as file:
    config = json.load(file)

radius_m = float(config["radius_m"])
```

Why it Matters

The context manager closes the file correctly, including when an exception occurs.

Robotics Use

Store target names, thresholds, paths, or non-ROS utility settings outside the source

Writing Telemetry or Test Evidence

Persist results in a format that can be inspected later

```
import csv

with open("telemetry.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["time_s", "altitude_m"])
    writer.writerow([0.0, 1.2])
```

Use Cases

Record test measurements, tracking error, detection confidence, or mission-state transitions for later analysis.

System Design Note

High-rate robotics logging needs deliberate buffering and storage design. This simple example teaches the file-handling pattern, not a production telemetry architecture.

1 Python Refresher for Robotics

- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

Virtual Environments

One isolated Python toolbox for one project

System Python $\neq$ project virtual environment

Why

Different projects may require different package versions. Isolation reduces conflicts and protects the operating-system Python environment.

Mental Model

The environment is a project-specific toolbox containing a Python interpreter context and installed packages.

Reproducibility

The environment directory is normally recreated from documented commands and dependency files, not copied between computers or committed to Git.

Create and Use a Virtual Environment

A repeatable Ubuntu workflow

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python --version
python -m pip --version

deactivate
```

How to Verify

After activation, `python` and `python -m pip` should refer to the environment. Verify rather than assuming from the prompt alone.

ROS 2 Compatibility

Some ROS 2 Python packages depend on the system Python environment. Follow the internship's documented setup rather than creating arbitrary environments for every node.

Install Packages with pip

Install into the interpreter you intentionally selected

```
python -m pip install numpy
python -m pip install -r requirements.txt
python -m pip list
```

Why `python -m pip`?

It runs pip through the selected Python interpreter, reducing ambiguity when several Python installations exist.

Project Practice

Document required versions in the repository. A dependency file supports repeatable installation on the second computer and by the internship students.

Avoid

Do not install project packages into the system Python with `sudo pip`. Use the documented environment and package-management strategy.

1 Python Refresher for Robotics

- Core Language Foundations
- Functions and Program Structure
- Object-Oriented Python for Robotics
- Errors, Files, and Configuration
- Python Environments and Dependencies
- NumPy and Basic Robotics Data Processing
- Logging, Debugging, and Integration

Why NumPy?

Robotics data often arrives as vectors, matrices, and images

```
import numpy as np

position_m = np.array([2.0, -1.0, 4.0])
velocity_mps = np.array([0.2, 0.0, -0.1])
```

Array Mental Model

A NumPy array is a structured grid of mostly same-type values. A vector is one-dimensional, a table is two-dimensional, and an image is commonly a two- or three-dimensional array.

Robotics Uses

Positions, velocities, point collections, camera frames, bounding boxes, masks, transformations, and batches of sensor samples.

Array Shape, Data Type, and Indexing

Understand the structure before processing the values

```
image = np.zeros((480, 640, 3), dtype=np.uint8)
print(image.shape)    # (height, width, channels)
print(image.dtype)    # uint8

pixel = image[120, 300]
```

shape

The size along each axis. Never assume image dimensions without checking.

dtype

The numeric representation. It affects range, precision, memory, and arithmetic behavior.

Vectorized Calculations

Express operations on complete arrays

```
measurements_m = np.array([2.4, 2.1, 1.9, 2.2])  
mean_m = measurements_m.mean()  
minimum_m = measurements_m.min()  
valid = measurements_m > 0.0  
filtered = measurements_m[valid]
```

What Happened?

NumPy calculated statistics and applied one condition to every measurement. The Boolean mask selected only valid values.

Engineering Rule

Vectorization can improve clarity and performance, but first verify array shape, units, coordinate frame, and data type.

Distance and Tracking Error

A small numerical pattern used throughout robotics

```
import numpy as np

desired_m = np.array([5.0, 0.0, -4.0])
measured_m = np.array([4.7, 0.2, -3.9])
error_vector_m = desired_m - measured_m
tracking_error_m = np.linalg.norm(error_vector_m)
```

Interpretation

Subtract positions to obtain an error vector. Its Euclidean norm gives one scalar distance between desired and measured positions.

Coordinate Discipline

Only subtract values expressed in compatible units and coordinate frames. NED and ENU vectors cannot be mixed without an explicit transformation.

Processing Detection Results

Filter observations before they enter the wider system

```
detections = [  
    {"label": "car", "confidence": 0.88},  
    {"label": "person", "confidence": 0.41},  
]
```

```
accepted = [d for d in detections  
             if d["confidence"] >= 0.60]
```

Pipeline Role

Python converts model output into structured observations: labels, confidence values, bounding boxes, timestamps, and frame identifiers.

Control Boundary

An accepted detection is still an observation. It should not directly command the

Logging Instead of Scattered Prints

Record what the program did and why

```
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

logger.info("Mission state: %s", "ORBIT")
logger.warning("No camera frame received")
logger.error("Configuration file is invalid")
```

Severity Levels

Use DEBUG for detailed diagnosis, INFO for normal milestones, WARNING for abnormal but recoverable situations, and ERROR for failed operations.

Logging Inside a ROS 2 Node

Use the framework logger for node-level evidence

```
self.get_logger().info("Mission ready")  
self.get_logger().warning("Waiting for odometry")  
self.get_logger().error("Invalid mission parameter")
```

Useful Log Content

Include the component, mission state, relevant value with units, and expected response. Avoid flooding the terminal from high-rate callbacks.

Project Connection

The orbit controller logs readiness, state transitions, and tracking error while ROS 2 manages node-specific output.

A Disciplined Debugging Workflow

Observe, isolate, test, and verify

1. Reproduce the problem with the exact command and environment.
2. Read the complete traceback or ROS 2 log message.
3. Identify the first relevant file, line, variable, topic, or callback.
4. Inspect values, types, array shapes, paths, and active environment.
5. Test one hypothesis with the smallest possible change.
6. Rerun the original scenario and record the verified result.

Avoid Random Repair

Repeated reinstallation, broad exception handling, and unrelated code changes destroy evidence and make the defect harder to isolate.

The same fundamentals appear inside event-driven robotics code

Compute first, validate second, publish third

```
altitude_m = -float(down_m)
if altitude_m < 0.0:
    self.get_logger().warning("Invalid altitude")
    return
```

Receive input, convert representation, validate assumptions, build a typed message, publish, and log unusual conditions.

How the Concepts Appear in the Internship Code

The refresher prepares you to read a complete robotics node

- ▶ **Variables and dictionaries:** mission parameters and limits.
- ▶ **Tuples and lists:** positions, setpoints, and message fields.
- ▶ **Conditions and state enumeration:** mission transitions.
- ▶ **Functions and methods:** validation, timing, commands, and callbacks.
- ▶ **Class inheritance:** the controller extends a ROS 2 Node.
- ▶ **Exceptions:** invalid parameters stop initialization clearly.
- ▶ **Imports and modules:** trajectory mathematics remain separate.
- ▶ **Logging:** readiness, state changes, and tracking error are observable.

Reading Strategy

Do not read the file as one long program. Identify the class state, initialization, communication interfaces, callbacks, helper methods, and repeating timer logic.

Knowledge Check

You are ready to continue when you can explain these ideas

- ▶ Which Python container best represents one position, a measurement history, and named configuration?
- ▶ Why should a distance function be separated from ROS 2 publication?
- ▶ What is the difference between a class and an object?
- ▶ Why does a ROS 2 Python node inherit from Node?
- ▶ When should an exception be caught, and when should it stop the program?
- ▶ Why use a virtual environment and `python -m pip`?
- ▶ What do NumPy `shape` and `dtype` tell us?
- ▶ Why is an AI detection not a direct flight-control command?

Python-to-Robotics Roadmap

Foundations become components of the complete project

Python fundamentals → structured functions and classes → NumPy data processing

→ ROS 2 nodes and callbacks → Gazebo/PX4 integration → AI observations

Python-to-Robotics Roadmap

Foundations become components of the complete project

Python fundamentals → structured functions and classes → NumPy data processing

→ ROS 2 nodes and callbacks → Gazebo/PX4 integration → AI observations

Next Step

The following ROS 2 tutorials will reuse these concepts to create packages, define nodes, exchange typed messages, process callbacks, and connect Python logic to the simulated drone system.